HW4

1. What did you observe first? Describe the initial behavior at level, in controlled tilts, in the complete sequence, and in a skipped-level test. Give at least two plausible early hypotheses and explain which test distinguished them

Initial Findings:
When it is held flat at the beginning a blue LED flashes briefly.
When the left edge of the board goes down, the green LED flashes briefly then turns off.
When the right edge goes down nothing happens to the LED.
When the far edge goes down nothing happens to the LED.
When the USB edge goes down nothing happens to the LED.
When the board goes down to any of the edges, the board screen briefly goes white.

I think that the hardware issue is to do with the shunts because the red LED isn't turning on. I ran into a similar issue with HW1 where UART wasn't sending because the shunt was in the wrong location. The fact that our boards were taken as well is another reason to believe this. I think that another issue could be some sort of delay happening. When I move the board left to right it seems to lag a bit, but this could be intended behavior.

2. What did the sensor measure? Include the five-pose raw ADC table and two worked roll-and-pitch calculations. Which reference values did you use, and why? Where did calculated and reported angles agree or disagree?

The calculated roll and pitch angles agreed with the firmware reported angles across all 5 of the poses when the calibration references and angle equations were used. The reference values were the calibrated X,Y, and Z readings. The level readings were very close to 0 degrees, and the angles were consistent. The left edge was negative roll, right edge positive roll, far edge was negative pitch and USB side was positive pitch. There were no significant disagreements between the calculated and reported angles.

Physical pose	Mean raw X	Mean raw Y	Mean raw Z	Largest change from level
Level	2027	2086	2879	Reference
Left edge down	1554	2113	2748	X: 473
Right edge down	2431	2075	2762	X:404
Far edge down	2023	1615	2739	Y:471
USB-side edge down	2014	2577	2694	Y:491

1. Left Edge Down

Raw values: $X = 1549,\ Y = 2114,\ Z = 2746$

Calibration: $X_0 = 2022,\ Y_0 = 2051,\ Z_0 = 2881$

$$dx = 1549 - 2022 = -473$$

$$dy = 2114 - 2051 = 63$$

$$dz = 2746 - 2881 + 512 = 377$$

\[Roll= \operatorname{atan2}(-473,377)\times57.2958 \]

$$Roll \approx -51.44^{\circ}$$

\[Pitch=\operatorname{atan2}(63,377)\times57.2958 \]

$$Pitch \approx 9.49^{\circ}$$

2. Far Edge Down

Raw values: $X = 2030,\ Y = 1612,\ Z = 2740$

Calibration: $X_0 = 2022,\ Y_0 = 2051,\ Z_0 = 2881$

$$dx = 2030 - 2022 = 8$$

$$dy = 1612 - 2051 = -439$$

$$dz = 2740 - 2881 + 512 = 371$$

\[Roll=\operatorname{atan2}(8,371)\times57.2958 \]

$$Roll \approx 1.24^{\circ}$$

\[Pitch=\operatorname{atan2}(-439,371)\times57.2958 \]

$$Pitch \approx -49.80^{\circ}$$

3. What happened during movement? Cite sample ticks from at least two transition runs.

Compare the timing of the raw axes and explain how you determined whether the readings in a row represented the same physical movement?

During the movement the X,Y, and Z changed as the board moved.

Run 1: motion_before.csv: From tick 526 to 531, Y fell from 1985 to 1797 counts, while pitch changed from −4.8° to −25.8°. The candidate changed from unknown to forward (F) at tick 531. Run 2: interrupted_before.csv: From tick 180 to 184, X fell from 1957 to 1613 counts. The candidate became left (L), and at tick 184 the stable pose became left and the sequence advanced to step 1.

I treated X, Y, and Z in each row as the same sampling cycle because the firmware starts their conversions together at 50 Hz. The axes weren't guaranteed to be sampled at the same time and the CSV doesn't have timestamps to verify this either.

4. When did a pose become stable? Show the candidate run lengths for at least one interrupted-pose test. Explain whether the observed stable transition satisfied the eight-consecutive-sample rule.

In the interrupted_before.csv, the candidate was left for 8 samples in a row (ticks 180-187). However, the pose changed to left at tick 184 which was after 5 samples not the 8 requested. This means that the firmware accepted the pose too early in the run

5. What were the firmware causes and repairs? Identify each root cause, cite its source location, explain the mechanism, and include a focused patch or before-and-after code excerpt. Connect each repair to an observation that it changes.

1. The stability counter accepted a pose too early. The sample counter wasn't able to reset when the candidate changed. This is shown in the interrupted_before.csv. The fix was to reset the counter when the candidate changed and to clear it when a candidate is unknown. Below is the code change.

```
if (state->candidate == POSE_UNKNOWN) {

    state->pending = POSE_UNKNOWN;

    state->pendingCount = 0U;

} else if (state->candidate != state->pending) {

    state->pending = state->candidate;

    state->pendingCount = 1U;

}
```

2. Another issue is that the firmware accepted poses without checking weather the board had stopped moving. This means that consecutive classifications could make a pose stable even when it was moving. The fix for this was to add start variables for the X,Y, and Z and to add counts for them. It also reset the stability counter whenever the movement exceeded the limits.
3. The third issue is that the Z axis was incorrectly scaled. This was an issue be when calculating the tilt angles it made the denominator too small. To fix this I added +512 to the line below.

   float dz = (float)((int32_t)z - state->levelZ + 512);

6. What caused the physical output discrepancy? Compare a commanded LED color with what you saw. Trace the signal through the board documentation; show the original and corrected physical connection and the resulting observation.

The physical output discrepancy was that the red boosterpack LED wasn't turning on. This is because the shunt on J5 was on pins 2-3, which selected the LCD backlight signal. This is on par with what I saw during the baseline tests. I didn't think much of it at first besides the red LED not turning on. When I looked into the documentation I saw that the shunt needed to move to pins 1-2 to select the red LED.

7. How do you know the system now meets the specification? Summarize before-and-after evidence for the full sequence, the skipped-level test, the interrupted-pose test, tilt calculations, movement timing, and visible RGB output. State any measurement uncertainty or remaining limitation.

The system now meets the specifications. For the full sequence before the steady only reached step 2 because the poses were not performed in order, after correcting it, it went to left, level, forward, level, right. The firmware now resets when an unexpected stable pose occurs. During the interrupted pose test it only reached step 3, but it now unlocks correctly. The tilt calculations were calculated with the formula and no external instruments were used to calculate these values. For movement timing, the samples are 20ms (50Hz) apart. For the visible RGB output, moving the J5 shunt from pins 2-3 to 1-2 fixed the red LED connection. The sequence also worked afterwards.

Problem solving and AI use:

1. Your debugging approach: How did you decide what to test first? Describe a point where an observation changed your hypothesis or made you design a better controlled test. What would you do differently if you started again?

The first thing I decided to test was the software because there were more bugs here than the hardware. I also knew that the hardware was likely going to take longer because the issue was

going to be looking through documentation. There was no point where I changed my hypothesis. I made a better controlled test when looking through the documentation, at first I thought the issue was going to be on the main ti, not the boosterpack, but started looking into boosterpack documentation after I realized the red LED was on there. If I started again, I would likely do the same thing. It would take me shorter because I would understand the instructions.

2. Your own work: Which measurements, physical board tests, calculations, source-code investigations, edits, and verification steps did you perform yourself? Be specific about your contribution, especially where you had to interpret results rather than simply run a command.

I did the hardware all by myself. I looked through the documentation for the ti at first and couldn't really find anything. From here, I looked through the documentation for the boosterpack and found what the issue was. I also had to interpret the results of the csv to an extent. I put the csv's into AI and I would determine if the AI was hallucinating or if the values made sense.

3. AI and other assistance: State whether you used AI. If you did, describe what you asked it to help with and which suggestions, explanations, or code changes you used. Give at least one concrete example of how you checked an AI suggestion against the board, trace, documentation, or source. If you rejected or changed a suggestion, explain why. If you did not use AI, say so and describe how you checked your own hypotheses.

I used AI for helping me debug the coding errors and help understanding the assignment. Because the instructions were written kind of poorly, I misunderstood how the board needed to tilt which led to me moving the board in improper ways. I also gave AI the code and asked it to fix any software bugs it found and gave it the instructions from the assignment. It was able to find some of the bugs easier than others. An example that it gave me was that it accepted a pose too early (code is above). I only rejected AI a few times because it got off topic.

4. Lessons from the process: Which issue was hardest to distinguish from another symptom? What evidence finally separated them? What general debugging habit will you carry into future embedded work?

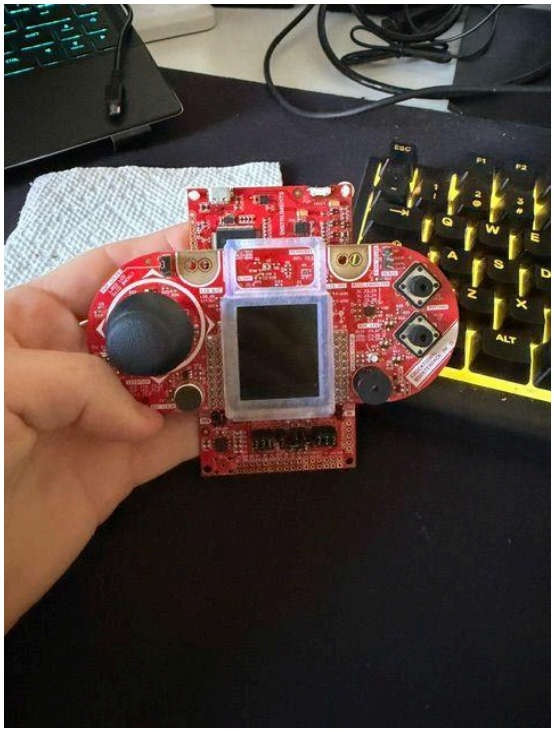
The hardest issue was the hardware bug. While I knew what the issue was, it took much longer to look through the documentation myself and find the issue. A good idea would be to tell AI the issue and give it the link to the documentation for it to figure out the issue itself. Evidence that separated them was the red LED not turning on and realizing something was turning it off. A general debugging habit is that I'll read the documentation and have a good idea of what can be a software bug versus a hardware one.

Hardware fix images:

Before picture:



After picture:


ESC
F1
F2
ALT